

# TokTickIT GitHub Workflow Guide

TokTickIT Quick Guide — CPE 334

Project board, Issues, and linking your Pull Request.

ตั้งบอร์ด สร้าง Issue และ link Pull Request

## What is in this guide

|                                                              |                                          |    |
|--------------------------------------------------------------|------------------------------------------|----|
| <i>Part 1 – Moving your card from Backlog to Done</i>        | <i>การเลื่อนการ์ดจาก Backlog ไป Done</i> | 2  |
| <i>Part 2 – Why linking your PR matters</i>                  | <i>ทำไมการ link PR ถึงสำคัญ</i>          | 4  |
| <i>Part 3 – Method 1: link it from the Development panel</i> | <i>link จากแถบ Development</i>           | 6  |
| <i>Part 4 – Method 2: typing a keyword</i>                   | <i>พิมพ์ keyword</i>                     | 7  |
| <i>Part 5 – How to check it worked</i>                       | <i>วิธีเช็คที่ทำสำเร็จ</i>               | 9  |
| <i>Part 6 – Linking your branch</i>                          | <i>การ link branch (Optional)</i>        | 9  |
| <i>Part 7 – Updating docs with no code</i>                   | <i>การอัปเดต docs ที่ไม่มีโค้ด</i>       | 10 |
| <i>Part 8 – Reviewing a Pull Request</i>                     | <i>การรีวิว Pull Request</i>             | 11 |
| <i>Part 9 – Agreements</i>                                   | <i>ข้อตกลง</i>                           | 14 |
| <i>Appendix – Set up your Project board</i>                  | <i>วิธีตั้ง Project board</i>            | 15 |

This is written around Lab 1, but the other labs work the same way.

คู่มือนี้เขียนจาก Lab 1 แต่เล่มอื่น ๆ ก็ทำแบบเดียวกัน

## Part 1 – Moving your card from Backlog to Done การเลื่อนการ์ดจาก Backlog ไป Done

The board has six statuses in this exact order. The definitions below are the ones from the labsheet, Section 12, so use them as the rule.

บอร์ดมี 6 สถานะ เรียงตามลำดับนี้เป๊ะ ๆ คำอธิบายข้างล่างยกมาจากเลขชี้หัวข้อ 12 ให้ยึดตามนี้

| Status    | When to use it                                                                                                                                                                                    |
|-----------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Backlog   | The Issue has been provided but has not yet been reviewed and understood.<br>Issue ถูกสร้างไว้แล้ว แต่ยังไม่ได้อ่านและยังไม่เข้าใจ                                                                |
| Specified | The Issue is understood and ready to implement.<br>อ่านแล้วเข้าใจ Issue นี้ และพร้อมลงมือทำ                                                                                                       |
| Started   | The feature branch has been created and implementation has begun.<br>สร้าง feature branch แล้ว และเริ่มลงมือเขียนแล้ว                                                                             |
| PR Review | A Pull Request to lab1-staging is open and the peer reviewer is checking it.<br>เปิด Pull Request เข้า lab1-staging แล้ว และเพื่อนที่รีวิวกำลังตรวจสอบอยู่                                        |
| Fixing    | Review changes are required or tests failed; corrections are being made on the same branch.<br>คนรีวิวขอให้แก้ หรือทดสอบไม่ผ่าน กำลังแก้อยู่บน branch เดิม                                        |
| Done      | The PR is approved, tests pass, it is merged into lab1-staging, and all acceptance criteria are satisfied.<br>PR ถูก approve ทดสอบผ่าน merge เข้า lab1-staging แล้ว และครบทุก acceptance criteria |

Three rules from the labsheet go with that table. Every Issue is added to the Project in Backlog. An Issue moves to Specified only after you have read and understood its requirements. And only the Issue you are actively implementing goes to Started.

มีกฎจากเลขชี้หัวข้อ 3 ข้อที่ใช้คู่กับตารางนี้ ทุก Issue ต้องถูกเพิ่มเข้า Project ที่ Backlog ก่อน จะย้ายไป Specified ได้ก็ต่อเมื่ออ่านและเข้าใจ requirement ของมันแล้ว และให้ย้ายเฉพาะ Issue ที่กำลังลงมือทำอยู่จริงไป Started เท่านั้น

After PR Review the card can go two ways. If it did not pass review it goes to Fixing, and from Fixing it goes back to PR Review once you have pushed the corrections. If it passed, it goes to Done.

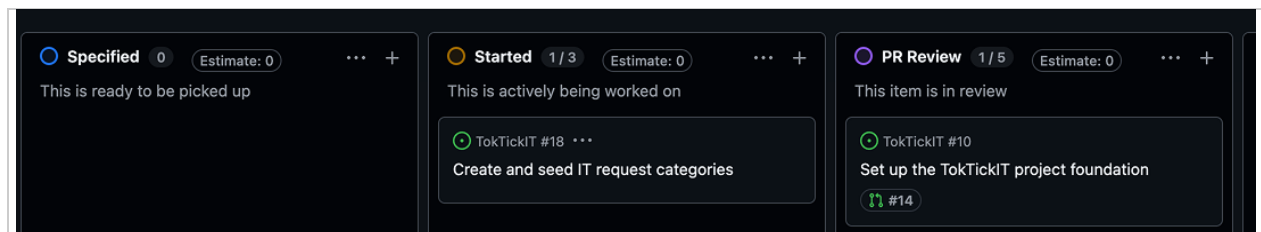
หลังจาก PR Review การ์ดไปได้สองทาง ถ้ารีวิวไม่ผ่านให้ไปที่ Fixing แล้วพอ push แก้เสร็จก็ย้อนกลับมา PR Review อีกรอบ ถ้าผ่านก็ไป Done

The same four moves, with real screenshots

สี่จังหวะเดิม พร้อมภาพจริง

Step 1. When you move a card to "Started," it is fine to link only the branch. There is no Pull Request yet at this point, and that is normal.

ขั้นที่ 1 ตอนย้ายการ์ดไปที่ "Started" จะ link แค่ branch เปล่า ๆ ก็ได้ ตอนนั้นยังไม่มี Pull Request ซึ่งเป็นเรื่องปกติ

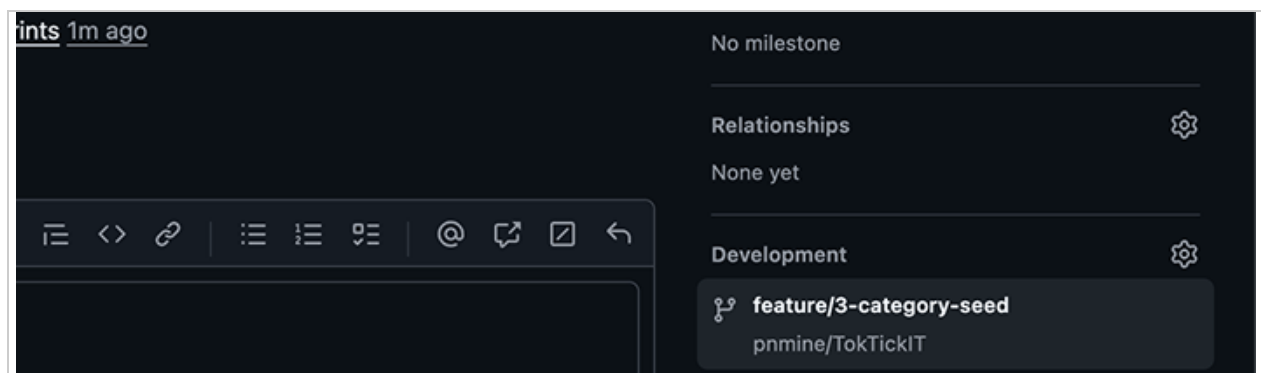


Issue #18 sits in "Started" with no PR number on the card yet.

Issue #18 อยู่ที่ "Started" โดยที่การ์ดยังไม่มีเลข PR

To link the branch: open the Issue, look at the right sidebar for "Development," click the gear icon beside it, and pick the branch you made for this Issue. Once it is linked, the branch name shows there. **(Optional)**

วิธี link branch: เปิดหน้า Issue แล้วดูแถบด้านขวาตรงหัวข้อ "Development" กดไอคอนรูปเฟืองข้าง ๆ แล้วเลือก branch ที่สร้างไว้สำหรับ Issue นี้ พอ link แล้วชื่อ branch จะขึ้นมาโชว์ตรงนั้น



On the Issue page, the gear beside "Development" is where you link the branch. Here feature/3-category-seed is already linked.

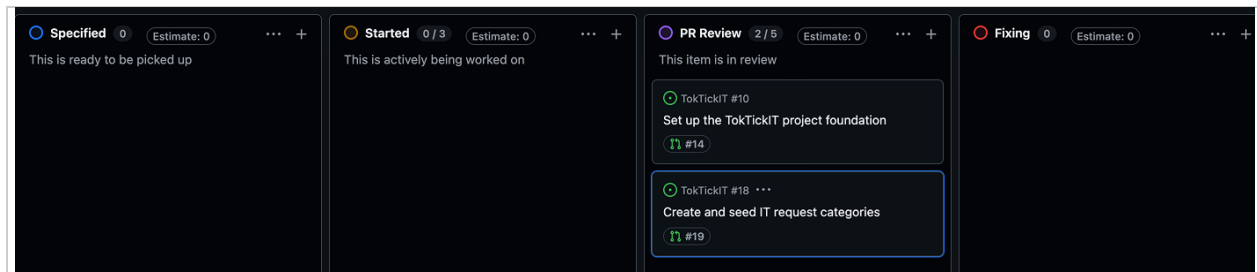
ในหน้า Issue ปุ่มเฟืองข้าง "Development" คือที่ใช้ link branch ในภาพนี้ feature/3-category-seed ถูก link ไว้แล้ว

Step 2. When your work is done and you open the Pull Request, **you still have to link that PR to the Issue.** Linking the branch earlier does not do this part for you.

ขั้นที่ 2 พอทำงานเสร็จและเปิด Pull Request แล้ว **ยังต้อง link PR นั้นเข้ากับ Issue อยู่ดี** การ link branch ไว้ตอนแรกไม่ได้ทำส่วนนี้ให้

Step 3. Move the Issue card to "PR Review" **only after the PR is linked.** You can tell it worked because the card now shows its PR number on it.

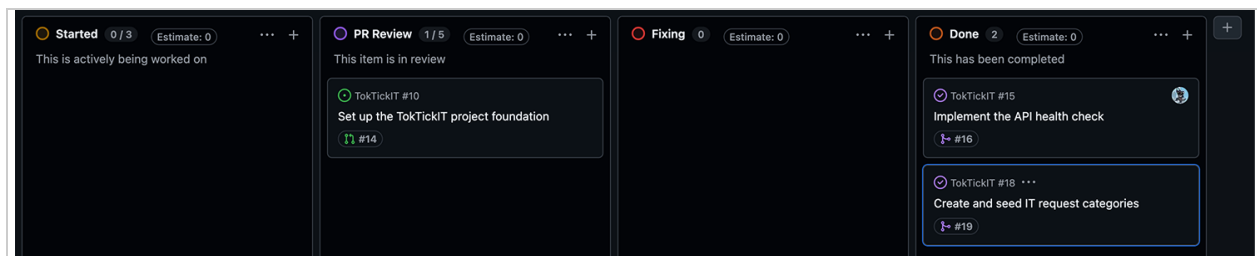
ขั้นที่ 3 ย้ายการ์ด Issue ไป "PR Review" แล้ว link PR เรียบร้อยแล้วเท่านั้น ถ้าทำถูก การ์ดจะมีเลข PR ติดอยู่บนการ์ดให้เห็น



Issue #18 has moved to "PR Review" and now carries its Pull Request #19 on the card.  
Issue #18 ย้ายมาที่ "PR Review" แล้ว และมี Pull Request #19 ติดอยู่บนการ์ด

Step 4. **After the reviewer merges the PR**, drag the card to "Done" and close the Issue yourself. The merge went into lab1-staging, so GitHub will not close it for you. (Once you drop the card in Done, refresh the page and you will usually see the project close the Issue for you. If it does not, just close it by hand.)

ขั้นที่ 4 **หลังคนรีวิว merge PR แล้ว** ให้ลากการ์ดไป "Done" แล้วปิด Issue เอง เพราะ merge เข้า lab1-staging GitHub จะไม่ปิดให้ (พอลากการ์ดลง Done แล้วลอง refresh หน้า ส่วนใหญ่ project จะปิด Issue ให้เอง ถ้าไม่ปิดก็กดปิดเองได้เลย)



Issue #18 in "Done" after the merge, still showing PR #19.  
Issue #18 อยู่ที่ "Done" หลัง merge และยังโชว์ PR #19 อยู่

## Part 2 – Why linking your PR matters **ทำไมการ link PR ถึงสำคัญ**

Every Issue you work on needs to be connected to the Pull Request that finished it. That link is how teams, and you, can look back at the board later and tell which PR did which Issue.

ทุก Issue ที่ทำ ต้องเชื่อมกับ Pull Request ที่ทำมันจนเสร็จ สิ่งนี้คือสิ่งที่ทำให้ทีมและตัวคุณเองย้อนกลับมาดูบอร์ดทีหลังแล้วรู้ว่า PR ไหนทำ Issue ไหน

GitHub can also close the Issue for you when the PR is merged. **But that only happens when the PR is merged into the repository's default branch** (the branch marked "default" on GitHub, usually main).

GitHub ปิด Issue ให้เองได้ตอน PR ถูก merge แต่จะเกิดขึ้นเฉพาะตอนที่ merge เข้า default branch ของ repo เท่านั้น (branch ที่ GitHub คิดป๊ายว่า "default" ปกติคือ main)

### What that means in Lab 1

In Lab 1 you merge your feature branch into lab1-staging, not into main. lab1-staging is not the default branch, so your Issue will still be Open after you merge. Nothing is broken.

ใน Lab 1 คุณ merge feature branch เข้า lab1-staging ไม่ได้เข้า main และ lab1-staging ไม่ใช่ default branch ดังนั้นหลัง merge แล้ว Issue จะยังเป็น Open อยู่ ไม่ได้พังหรือทำผิดอะไร

Close the Issue yourself and move the card to Done. The link you made is still saved on the PR, which is the part that counts.

ให้ปิด Issue เองแล้วย้ายการ์ดไป Done ส่วนลิงก์ที่ทำไว้ยังถูกบันทึกอยู่บน PR ซึ่งเป็นส่วนที่นับ

### The one thing to remember สิ่งเดียวที่ต้องจำ

Linking a branch to an Issue (the branch name you sometimes see under "Development") is nice to have, but it is not the thing we check.

การ link branch เข้ากับ Issue (ชื่อ branch ที่บางทีเห็นใต้ "Development") มีก็ดี แต่ไม่ใช่สิ่งที่เราตรวจ

The thing we check is the PULL REQUEST linked to its ISSUE. That is what this guide is about.

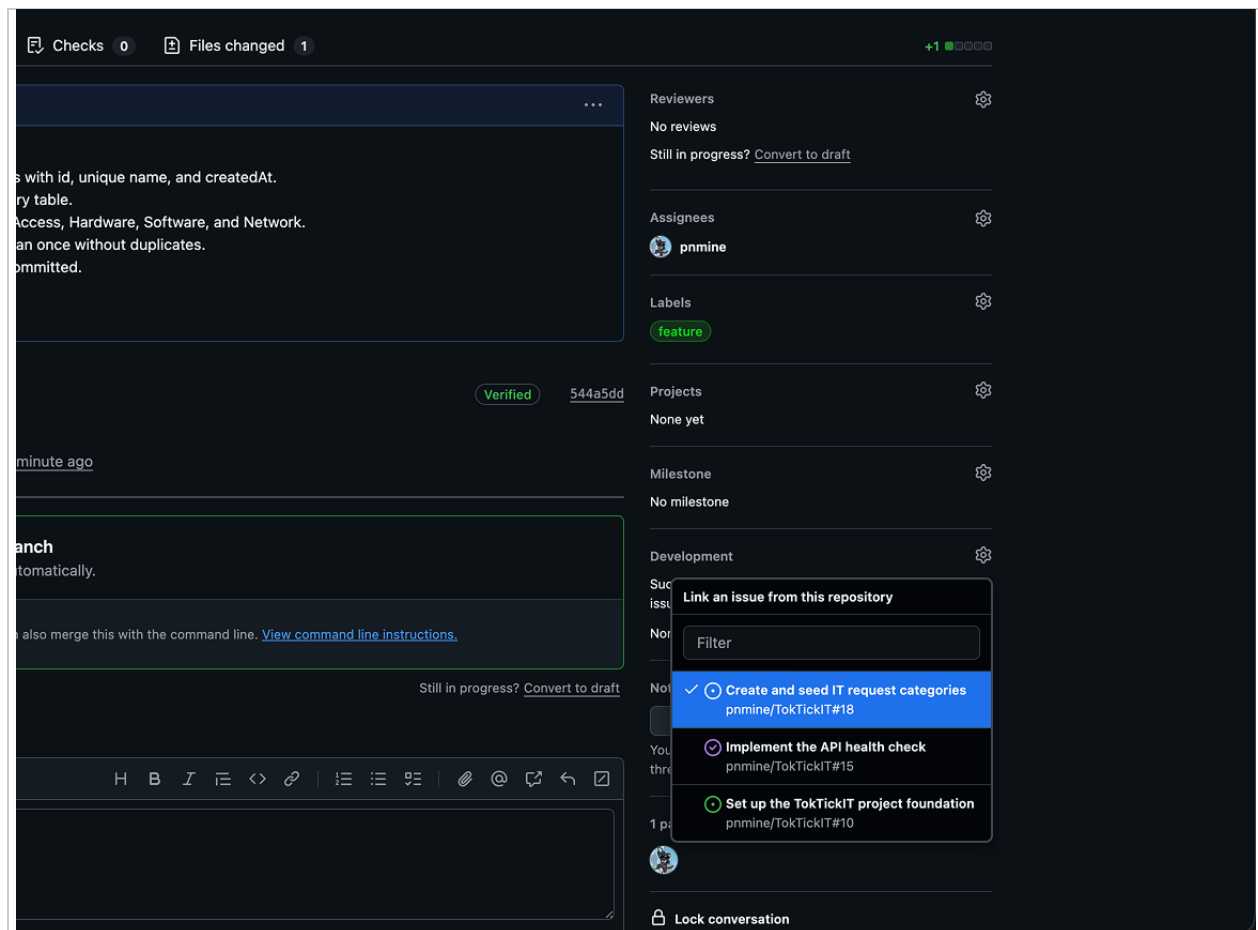
สิ่งที่เราตรวจคือ PULL REQUEST ที่ link กับ ISSUE ของมัน และนั่นคือเรื่องที่คุณมีมือนี้ผูกถึง

### Part 3 – Method 1: link it from the Development panel link จากแถบ Development

This is the easiest way, and it still works days after you opened the PR. But do it right after you create the PR. That is the habit we want, and it is how you avoid forgetting.

วิธีนี้ง่ายที่สุด และถึงเปิด PR ทิ้งไว้หลายวันแล้วก็ยังทำได้ แต่ให้ทำทันทีตอนสร้าง PR เสร็จเลย เป็นนิสัยที่เราอยากให้คุณ และจะได้ไม่ลืม

1. Open your Pull Request. เปิดหน้า Pull Request ของคุณ
2. On the right sidebar, find "Development." ที่แถบด้านขวา หาหัวข้อ "Development"
3. Click the gear icon ⚙️ next to it. คลิกไอคอนรูปเฟือง ที่อยู่ข้าง ๆ
4. Search for your Issue and click it to link. ค้นหา Issue ของคุณแล้วคลิกเพื่อ link



Click the gear icon next to "Development," then search and select your Issue.  
คลิกไอคอนรูปเฟืองข้าง "Development" แล้วค้นหาและเลือก Issue ของคุณ

## Part 4 – Method 2: typing a keyword พิมพ์ keyword

You may have seen people type a keyword and the Issue number in the PR description:

หลายคนน่าจะเคยเห็นการพิมพ์ keyword ตามด้วยเลข Issue ในคำอธิบาย PR แบบนี้:

Closes #18   Resolves #18   Fixes #18

(Keywords must be typed in English – do not translate them. / คำเหล่านี้ต้องพิมพ์เป็นภาษาอังกฤษเท่านั้น ห้ามแปล)

As you type #, GitHub shows an autocomplete list of your Issues. Pick the right one from the list instead of typing the number by hand.

พอพิมพ์ # ระบบจะขึ้นลิสต์ autocomplete ของ Issue ให้เลือก แนะนำให้เลือกจากลิสต์แทนการพิมพ์เลขเอง

The top screenshot shows the GitHub Pull Request interface. At the top, it says "base: lab1-staging" and "compare: feature/3-category-seed" with a green checkmark indicating "Able to merge". Below this, there's a section "Add a title \*" with the text "feat: add category-seed". Underneath is "Add a description" with a "Write" tab selected. A dropdown menu for "Resolve #" is open, showing a list of issues: "#18 Create and seed IT request categories" (selected), "#14 feat: add project-foundation", "#15 Implement the API health check", "#17 Lab1 staging", and "#16 feat: add /api/health endpoint".

The bottom screenshot shows the same interface after selection. The "Resolve #" dropdown is now closed, and the description text area contains the following text: "Resolve #18", "Add A Prisma Category model exists with id, unique name, and createdAt.", "Add A migration creates the Category table.", "Add The seed inserts Account and Access, Hardware, Software, and Network.", "Add The seed is safe to run more than once without duplicates.", and "Add Database credentials are not committed." At the bottom right, there is a green button labeled "Create pull request".

Type "Resolve #" (or Closes / Fixes) and pick your issue from the list that pops up.

พิมพ์ "Resolve #" (หรือ Closes / Fixes) แล้วเลือก Issue จากลิสต์ที่ขึ้นมา

## The keyword alone does not link anything in Lab

ใน Lab ฝึก keyword อย่างเดียวไม่ได้ link อะไรเลย

A keyword only creates the link when your PR targets the repository's default branch. In Lab 1 your PR targets lab1-staging, so GitHub treats the keyword as a plain mention.

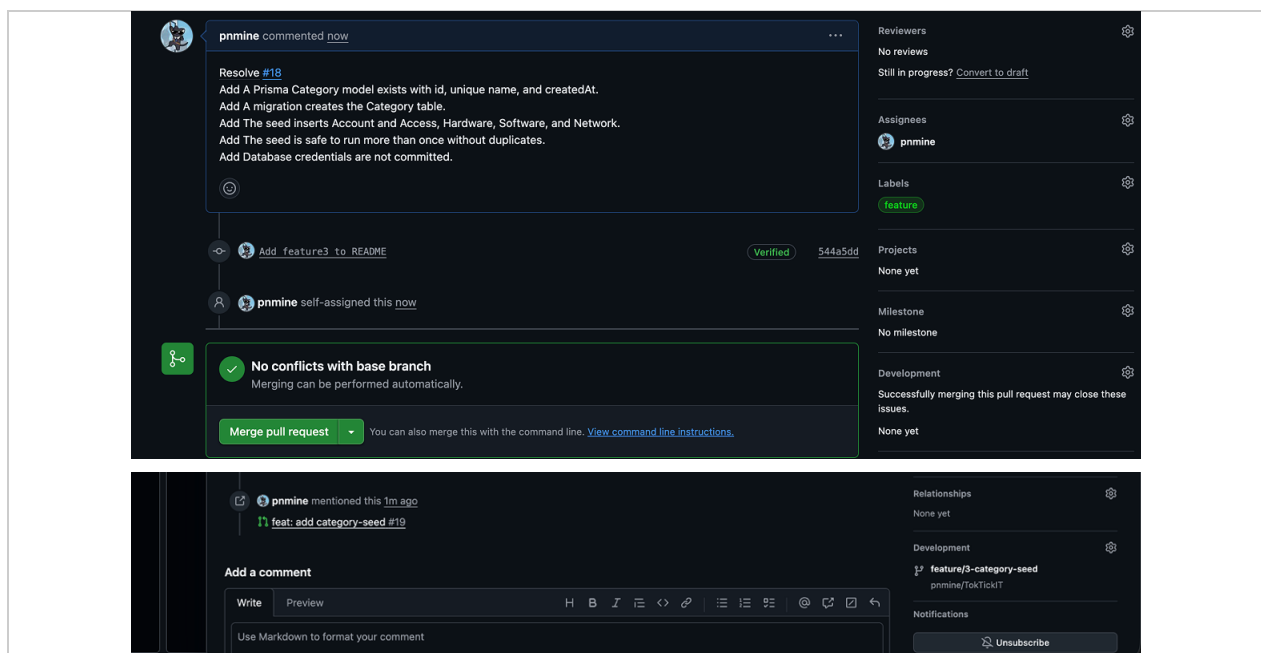
keyword จะสร้าง link ให้ก็ต่อเมื่อ PR ชี้ไปที่ default branch ของ repo เท่านั้น แต่ใน Lab 1 PR ของเราชี้ไปที่ lab1-staging GitHub เลยนับ keyword นั้นเป็นแค่การ mention เฉย ๆ

You can see it happen: the PR sidebar still says "none yet," the Issue says "mentioned this" instead of "linked," and the card on the board still has no PR number.

ดูได้จากของจริงเลย แถบขวาของ PR ยังขึ้นว่า "none yet" ส่วน Issue ขึ้นว่า "mentioned this" ไม่ใช่ "linked" และการ์ดบนบอร์ดก็ยังไม่มียุ่ PR

So type the keyword if you want, it makes your description clearer. But you still have to link it with Method 1 afterwards.

เพราะฉะนั้นจะพิมพ์ keyword ก็ได้ ทำให้คำอธิบายอ่านง่ายขึ้น แต่ยังไงก็ต้องไป link ด้วยวิธีที่ 1 อยู่ดี



After typing "Resolve #18": the PR sidebar shows none yet, and the Issue only says it was mentioned.  
หลังพิมพ์ "Resolve #18" แล้ว แถบขวาของ PR ขึ้น none yet ส่วน Issue ขึ้นแค่ว่าถูก mention

## Full keyword list รายการ keyword ทั้งหมด

close, closes, closed

fix, fixes, fixed

resolve, resolves, resolved

<https://docs.github.com/en/issues/tracking-your-work-with-issues/using-issues/linking-a-pull-request-to-an-issue>

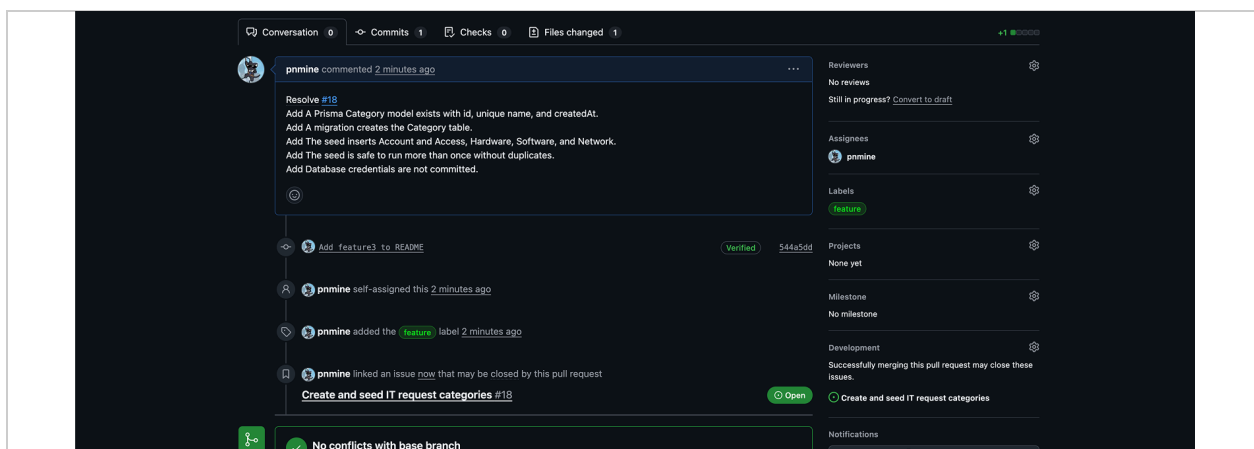
## Part 5 – How to check it worked วิธีเช็คที่ทำสำเร็จ

The check that works for both methods is the right sidebar of your PR. Under "Development" it should say "Successfully merging this pull request may close these issues," followed by your Issue.

วิธีเช็คที่ใช้ได้กับทั้งสองวิธีคือดูแถบขวาของ PR ตรงหัวข้อ "Development" ต้องขึ้นว่า "Successfully merging this pull request may close these issues" แล้วตามด้วย Issue ของคุณ

If it says "none yet," the Issue is not linked. Go back to Method 1 and link it from the Development panel. A keyword in the description is not enough on its own.

ถ้าตรงนั้นขึ้นว่า "none yet" แปลว่ายังไม่ได้ link ให้กลับไปทำวิธีที่ 1 link จากแถบ Development ให้เรียบร้อย แต่พิมพ์ keyword ในคำอธิบายยังไม่พอ



Linked from the Development panel here, so the "linked an issue" line shows. Check the sidebar on the right either way.  
PR นี้ link จากแถบ Development เลยมีบรรทัด "linked an issue" ขึ้น ส่วนแถบขวาคือจุดที่ต้องเช็คไม่ว่าจะใช้วิธีไหน

Once the sidebar shows your Issue, you can merge as usual. Remember that in Lab 1 the merge goes into lab1-staging, so merging alone will not close the Issue. Drag the card to Done and refresh, or close it by hand.

พอแถบขวาขึ้น Issue ของคุณแล้ว merge ได้ตามปกติ แต่อย่าลืมว่าใน Lab 1 merge เข้า lab1-staging แต่ merge จะยังไม่ปิด Issue ให้ลากการ์ดไป Done แล้ว refresh หรือกดปิดเอง

## Part 6 – Linking your branch การ link branch (Optional)

Link the branch when you move the card to "Started," right before you begin implementing. It puts the branch name under "Development" on the Issue page, so **anyone looking at the Issue can see the work has begun.**

ให้ link branch ตอนย้ายการ์ดไปที่ "Started" ก่อนเริ่ม implement มันจะทำให้ชื่อ branch ไปโผล่ใต้ "Development" ในหน้า Issue ใครเปิดดู Issue ก็จะรู้ว่างานเริ่มแล้ว

Linking the branch is a separate thing from linking the PR. Doing it does not replace Part 3, **and you still link the PR when the work is finished.**

การ link branch เป็นคนละเรื่องกับการ link PR ทำแล้วไม่ได้แทนส่วนที่ 3 พองานเสร็จก็**ยังต้อง link PR อยู่ดี**

## Part 7 – Updating docs with no code การอัปเดต docs ที่ไม่มีโค้ด

From now on you will be editing .md files that have nothing to do with code, such as the lab report, your reflection, and ai-use.md. This is what to do in each case.

ต่อจากนี้จะมีการแก้ไขไฟล์ .md ที่ไม่เกี่ยวกับโค้ดเลย เช่น รายงานของแล็บ reflection และ ai-use.md แต่ละกรณีให้ทำแบบนี้

| Situation                                                                                                               | What to do                                                                                                                                                                                                                                                                                         |
|-------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>The code for the Issue is still in progress</b><br>โค้ดของ Issue นั้นยังทำอยู่                                       | Edit the docs on the same feature branch. They go in with the same Pull Request. Do not open a second branch for them.<br>แก้ docs บน feature branch เดิมได้เลย แล้วให้มันไปพร้อม Pull Request เดียวกัน ไม่ต้องเปิด branch ที่สอง                                                                  |
| <b>The code is already merged and the docs change is a big one</b><br>โค้ดถูก merge ไปแล้ว และ docs ที่จะแก้ไขอะพอสมควร | Open a new branch just for the docs, for example docs/lab1-report, and open a Pull Request for it. The history then shows exactly what was written and when.<br>เปิด branch ใหม่สำหรับ docs โดยเฉพาะ เช่น docs/lab1-report แล้วเปิด Pull Request ของมัน ประวัติจะได้เห็นชัดว่าเขียนอะไรเพิ่มตอนไหน |
| <b>A very small edit, a typo or a broken link</b><br>แก้เล็กน้อยจริง ๆ เช่น คำผิดหรือลิงก์เสีย                          | Still a branch and still a Pull Request, but treat it as a fast lane. Your reviewer can read it and approve in a minute.<br>ก็ยิ่งเปิด branch และเปิด Pull Request เหมือนเดิม แต่ถือเป็นทางด่วน คนรีวิวอ่านแล้ว approve ได้ในนาทีเดียว                                                             |

name a docs-only branch docs/<lab>-<topic>, for example docs/lab1-ai-use. If the docs belong to an Issue, link the PR to that Issue as usual. If there is no Issue for it, say so in one line in the PR description.

ตั้งชื่อ branch ที่มีแค่ docs เป็น docs/<lab>-<topic> เช่น docs/lab1-ai-use ถ้า docs นั้นเป็นของ Issue ใดๆ ก็ link PR เข้ากับ Issue นั้นตามปกติ ถ้าไม่มี Issue ให้เขียนบอกสักบรรทัดในคำอธิบาย PR

### Everything reaches lab1-staging through a Pull Request

#### ทุกอย่างเข้า lab1-staging ผ่าน Pull Request เท่านั้น

The labsheet says do not develop directly on main or lab1-staging, and a docs commit is no exception. If you never push straight to lab1-staging, you cannot get this wrong.

ห้ามทำงานตรง ๆ บน main หรือ lab1-staging และ commit ที่เป็น docs ก็ไม่ได้ยกเว้น ถ้าไม่เคย push ตรงเข้า lab1-staging เลย ก็จะไม่มีการทำผิดข้อนี้

A Pull Request also gives you the two things a docs change still deserves: someone reads it, and the history shows what changed.

การเปิด Pull Request ยังให้สออย่างทำงาน docs ควรได้เหมือนกัน คือมีคนอ่าน และมีประวัติว่าทำอะไรไปบ้าง

## Part 8 – Reviewing a Pull Request การรีวิว Pull Request

A review is a conversation between two people. Here is what each side does.

การรีวิวคือการคุยกันระหว่างสองคน ฝ่ายไหนต้องทำอะไรบ้าง ดูตามนี้

**If you are the reviewer:** ถ้าคุณเป็นคนรีวิว

1. Open the PR and read the "Files changed" tab. Check the code against the acceptance criteria on the Issue, not just whether it runs.

1. เปิด PR แล้วดูแท็บ "Files changed" อ่านโค้ดเทียบกับ acceptance criteria ใน Issue ไม่ใช่ดูแค่ว่ารันได้

2. To comment on a specific line, hover over it and click the blue plus, write your comment, then click "Start a review." Add as many line comments as you need.

2. ถ้าจะ comment ที่บรรทัดไหน ให้เอาเมาส์ชี้ที่บรรทัดนั้นแล้วกดปุ่มบวกสีน้ำเงิน พิมพ์ comment แล้วกด "Start a review" จะ comment ที่บรรทัดก็ได้

3. When you are finished, click "Review changes" at the top right and choose one of the three options below.

3. พอดูครบแล้ว กด "Review changes" ที่มุมขวาบน แล้วเลือกหนึ่งในสามตัวเลือกข้างล่างนี้

| Option ตัวเลือก | Use it when... ใช้ตอนไหน                                                                                                                              |
|-----------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
| Comment         | you have questions or remarks but you are not making a decision yet.<br>มีคำถามหรือข้อสังเกต แต่ยังไม่ตัดสินใจว่าผ่านหรือไม่ผ่าน                      |
| Approve         | The code meets the acceptance criteria and you are happy to merge it.<br>โค้ดตรงตาม acceptance criteria แล้ว และคุณโอเคที่จะ merge                    |
| Request changes | Something has to be fixed before this can be merged. Say clearly what has to change.<br>มีอะไรที่ต้องแก้ก่อนถึงจะ merge ได้ ให้บอกให้ชัดว่าต้องทำอะไร |

4. If you approved, you are also the one who clicks "Merge pull request." Do not leave it for the author.

4. ถ้า approve แล้ว คุณเป็นคนกด "merge pull request" เองด้วย อย่าปล่อยให้คนอื่นเปิด PR กด

5. If you requested changes, tell your partner so they know to start fixing.

5. ถ้ากด Request changes ให้บอกเพื่อนด้วย เขาจะได้รู้ว่าต้องเริ่มแก้แล้ว

## If you are the author:

ถ้าคุณเป็นคนเปิด PR:

1. Reply to every comment. Write what you changed, or say why you disagree. An approval with silence underneath it is not a review.

1. ตอบกลับทุก comment เขียนว่าแก้ไขอะไรไป หรือบอกว่าทำไมไม่เห็นด้วย ถ้ามีแต่ approve แล้วเงียบ อันนั้นไม่นับว่ารีวิว

2. Move your card to Fixing while you work on the changes, and push to the same branch. The PR updates itself, you do not open a new one.

2. ระหว่างแก้ ให้ย้ายการ์ดไปที่ Fixing แล้ว push ขึ้น branch เดิม PR จะอัปเดตเอง ไม่ต้องเปิด PR ใหม่

3. When the fix is pushed, reply on the thread and move the card back to PR Review so your reviewer knows to look again.

3. พอ push แก้เสร็จ ให้ตอบใน thread แล้วย้ายการ์ดกลับไป PR Review เพื่อนจะได้รู้ว่ากลับมาดูได้แล้ว

4. Resolve a conversation only after you have replied to it and actually fixed it.

4. กด Resolve conversation ก็ต่อเมื่อตอบกลับแล้วและแก้จริงแล้วเท่านั้น

## Part 9 – Agreements ข้อตกลง

### Agreed in the lab session

The REVIEWER, not the PR author, is the one who clicks "merge pull request" after approving.

คนที่ REVIEW เป็นคนกด "merge pull request" หลัง approve ไม่ใช่คนเปิด PR

If your reviewer leaves a comment, reply to it. Do not just take the approval and merge without saying anything.

ถ้าคนรีวิว comment มา ให้ตอบกลับด้วย อย่ารีบ approve แล้ว merge ไปเฉยๆ โดยไม่พูดอะไรเลย

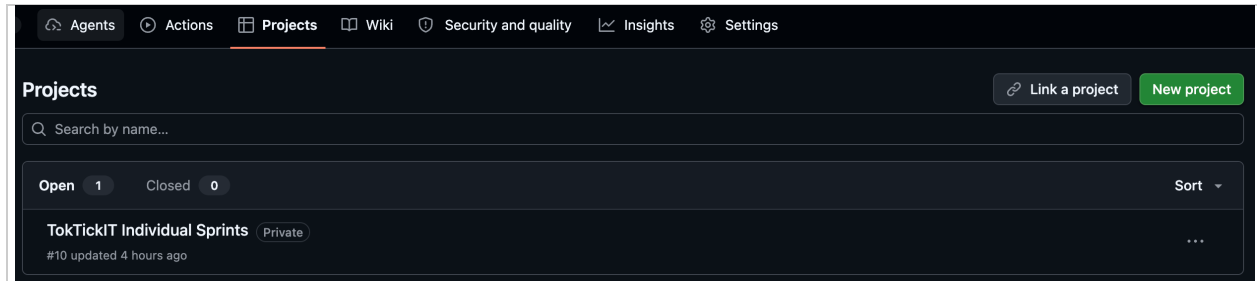
## Appendix – Set up your Project board วิธีตั้ง Project board

You only do this once per sprint, and most of you have done it already. Keep it for the next lab, since every lab uses a board with the same six columns.

ส่วนนี้ทำครั้งเดียวต่อหนึ่ง sprint และหลายคนน่าจะทำเป็นกันแล้ว เก็บไว้ใช้ตอนแล็บหน้าได้ เพราะทุกแล็บใช้บอร์ดที่มี 6 คอลัมน์เหมือนกัน

Step 1. Open your repository, go to the "Projects" tab, and click "New project."

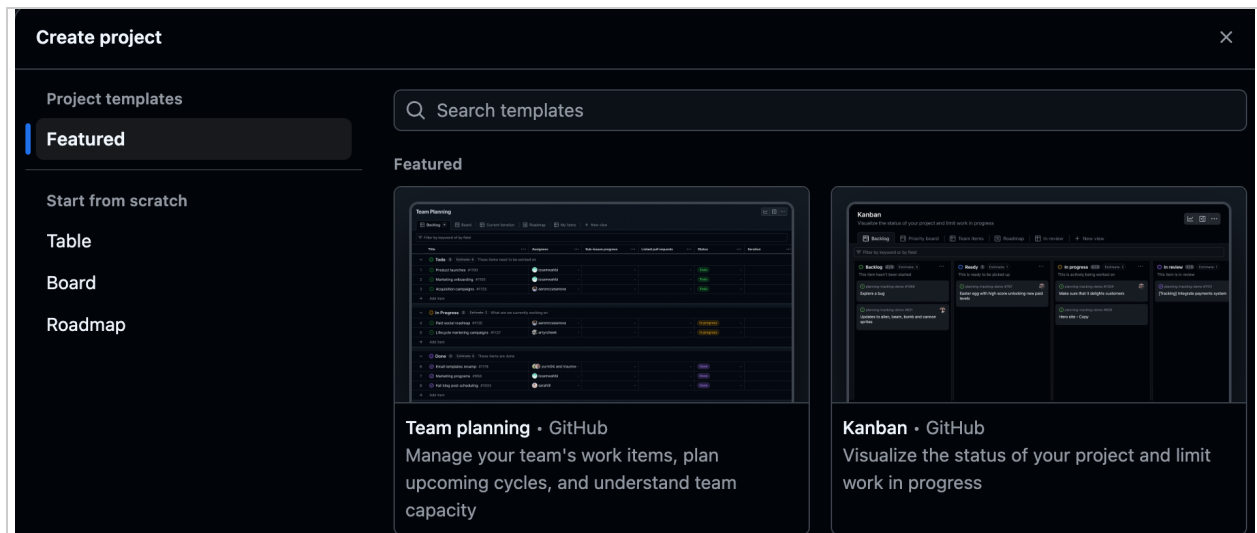
ขั้นที่ 1 เปิด repository ของคุณ ไปที่แท็บ "Projects" แล้วกด "New project"



The Projects tab of your repository.  
แท็บ Projects ในหน้า repository ของคุณ

Step 2. Pick the "Kanban" template. Do not pick Team planning, Feature release, or Bug tracker.

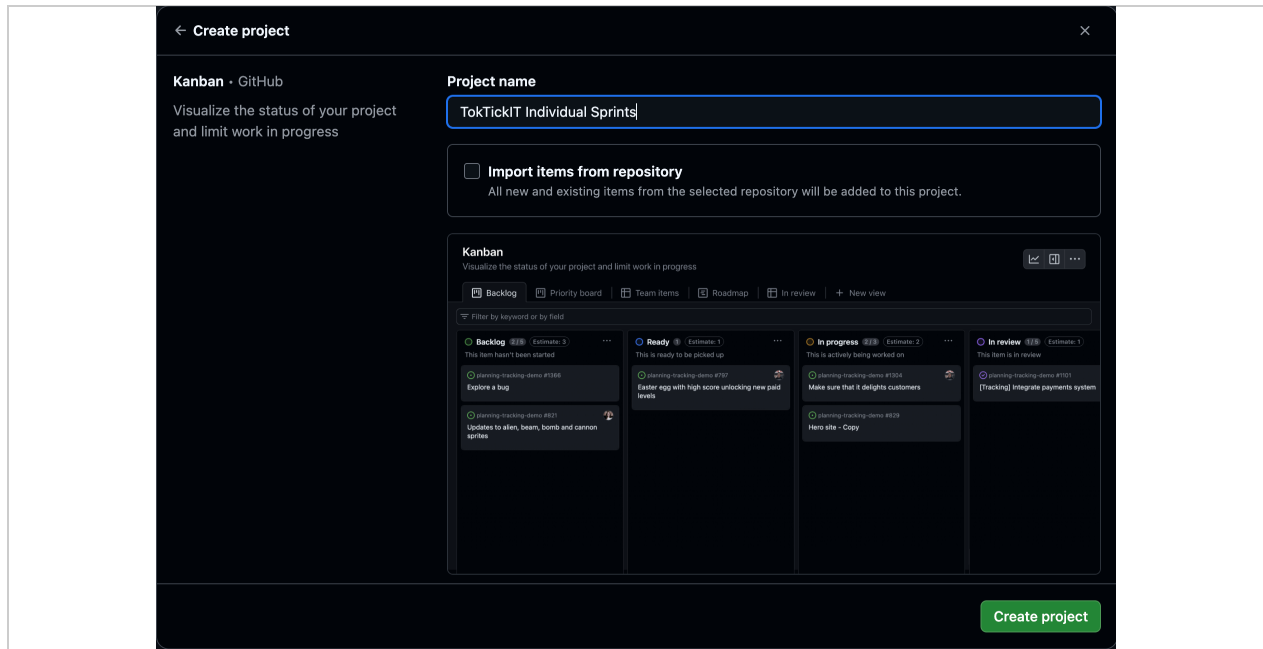
ขั้นที่ 2 เลือก template ชื่อ "kanban" อย่าเลือก Team planning, Feature release หรือ Bug tracker



Choose Kanban from the Featured templates.  
เลือก Kanban จากหมวด Featured

Step 3. Name it TokTickIT Individual Sprints, leave "Import items from repository" unchecked, and click "Create project."

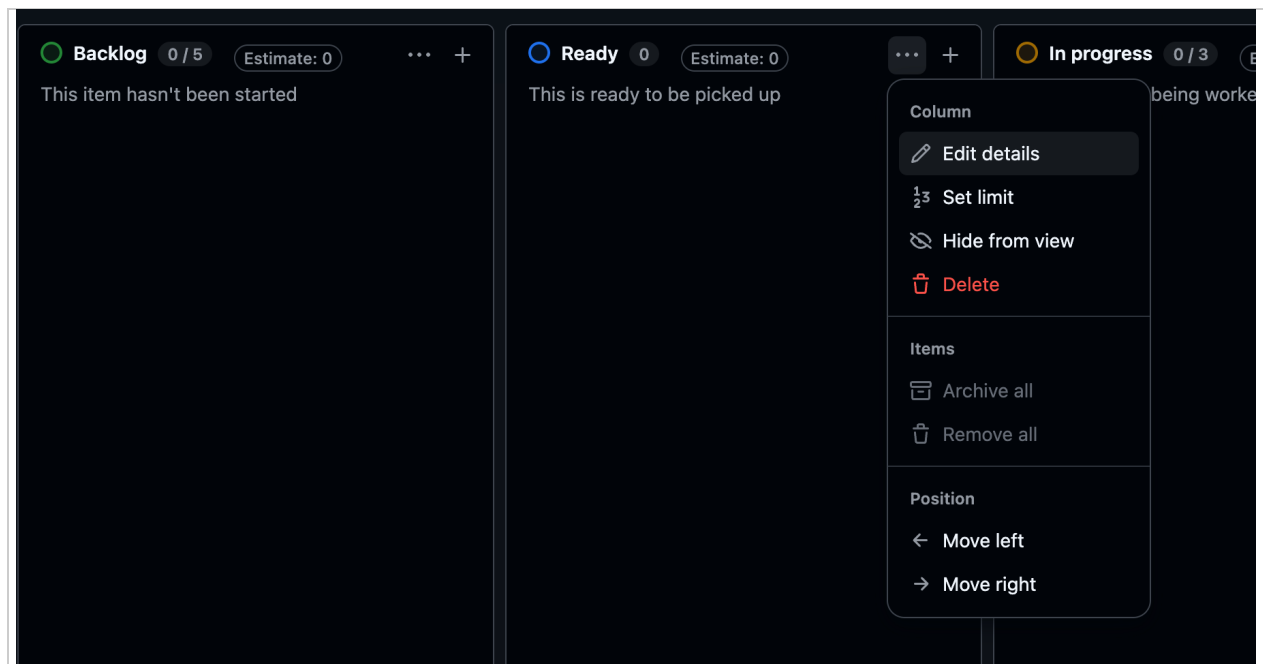
ขั้นที่ 3 ตั้งชื่อว่า TokTickIT Individual Sprints ไม่ต้องติ๊ก "Import items from repository" แล้วกด "Create project"



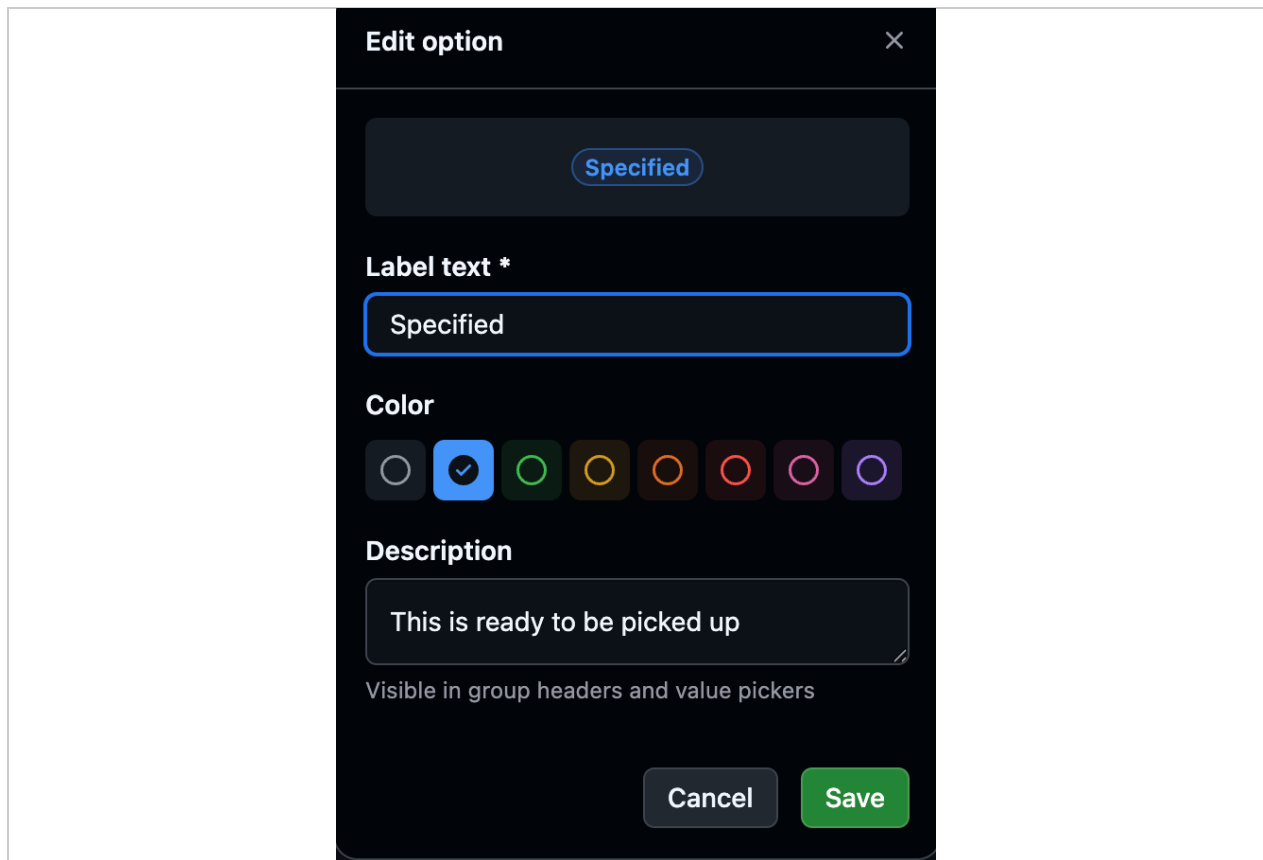
Type the project name, then Create project.  
พิมพ์ชื่อ project แล้วกด Create project

Step 4. Rename the columns. The template names are not the ones we use, so click the three dots on a column, choose "Edit details," change the Label text, and Save.

ขั้นที่ 4 เปลี่ยนชื่อคอลัมน์ ชื่อที่ template ให้มาไม่ตรงกับที่เราใช้ ให้กดจุดสามจุดบนคอลัมน์ เลือก "Edit details" แก้ชื่อ Label text แล้วกด Save



Three dots on the column, then Edit details.  
กดจุดสามจุดบนคอลัมน์ แล้วเลือก Edit details



Change Label text and Save.  
แก้ไข Label text แล้วกด Save

Rename them like this:

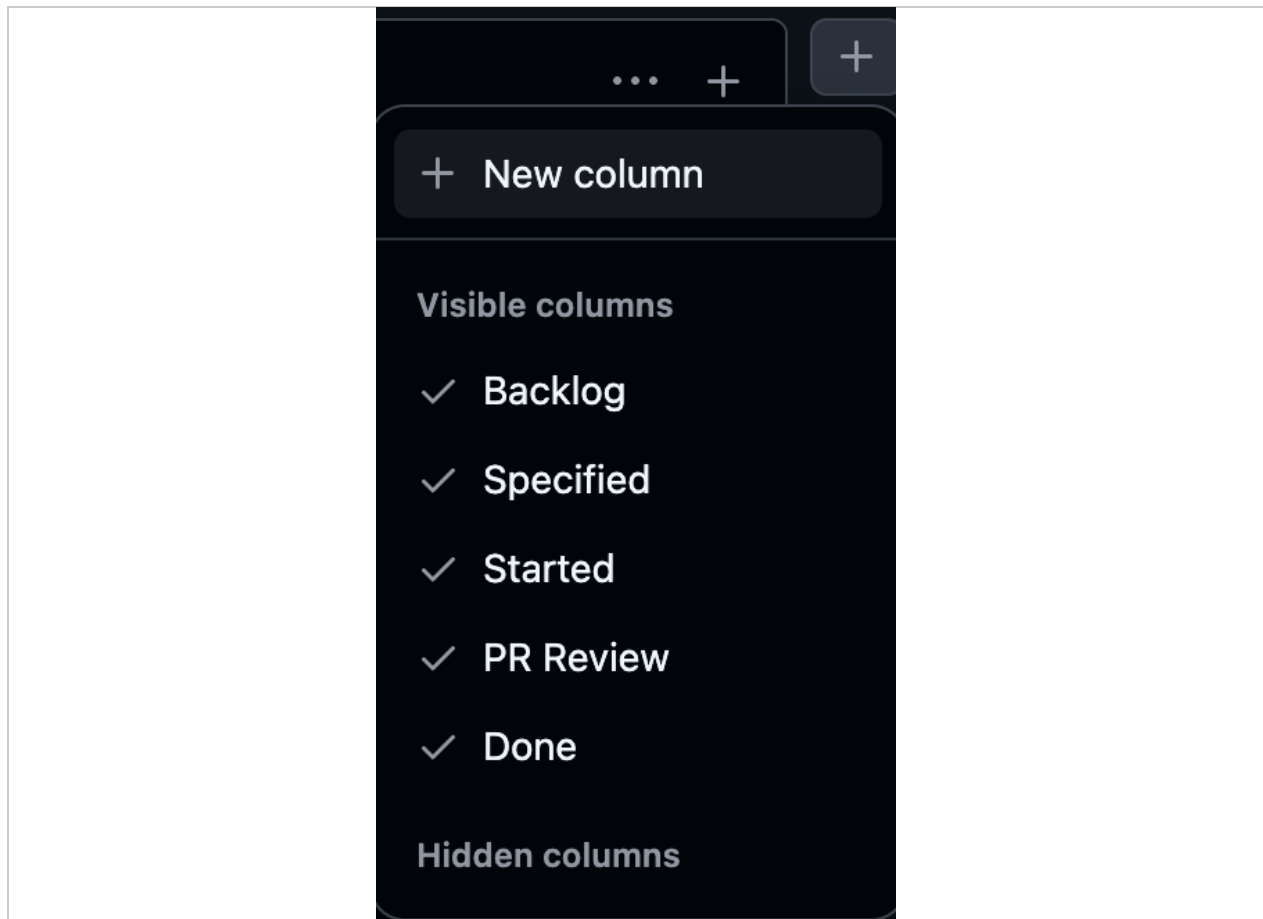
เปลี่ยนชื่อตามนี้:

| Kanban template name | TokTickIT name      |
|----------------------|---------------------|
| Backlog              | Backlog (no change) |
| Ready                | Specified           |
| In progress          | Started             |
| In review            | PR Review           |
| Done                 | Done (no change)    |

The same menu has "Set limit," which sets the work-in-progress limit you see on the column header (for example 0 / 5). /  
เมนูเดียวกันมี "Set limit" ไว้ตั้งลิมิตงานที่ทำพร้อมกัน ซึ่งก็คือเลข 0 / 5 บนหัวคอลัมน์

Step 5. Add the missing column. We use six columns and the template only gives five, so click the plus button on the far right of the board, choose "new column," and name it Fixing. Put it between PR Review and Done.

ขั้นที่ 5 เพิ่มคอลัมน์ที่ขาดไป เราใช้ 6 คอลัมน์ แต่ template ให้มา 5 ให้คลิกปุ่มบวกที่ขวาสุดของบอร์ด เลือก "new column" แล้วตั้งชื่อว่า Fixing วางไว้ระหว่าง PR Review กับ Done

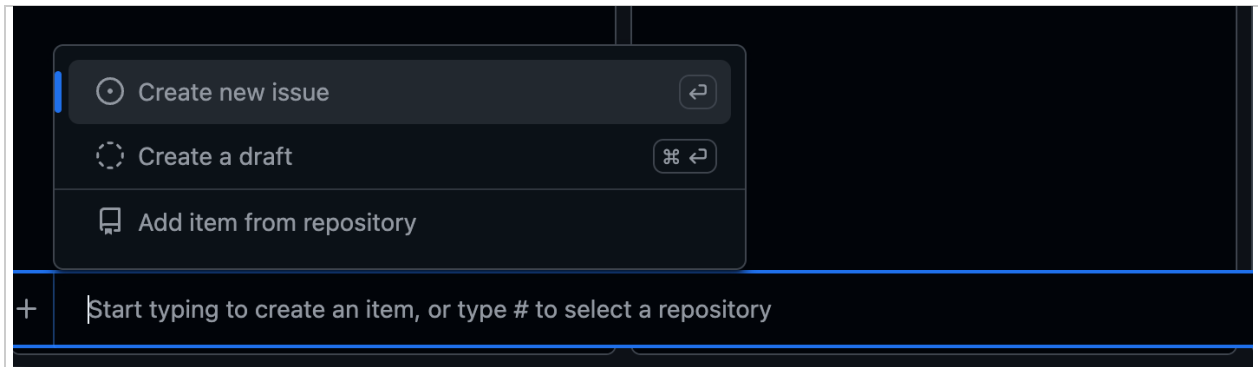


new column, then name it Fixing.  
กด new column แล้วตั้งชื่อว่า Fixing

When you are done the board reads: Backlog, Specified, Started, PR Review, Fixing, Done.  
พอเสร็จแล้วบอร์ดจะเรียงเป็น Backlog, Specified, Started, PR Review, Fixing, Done

Step 6. Add your Issues. At the bottom of the Backlog column click "+ Add item," then choose "Create new issue."

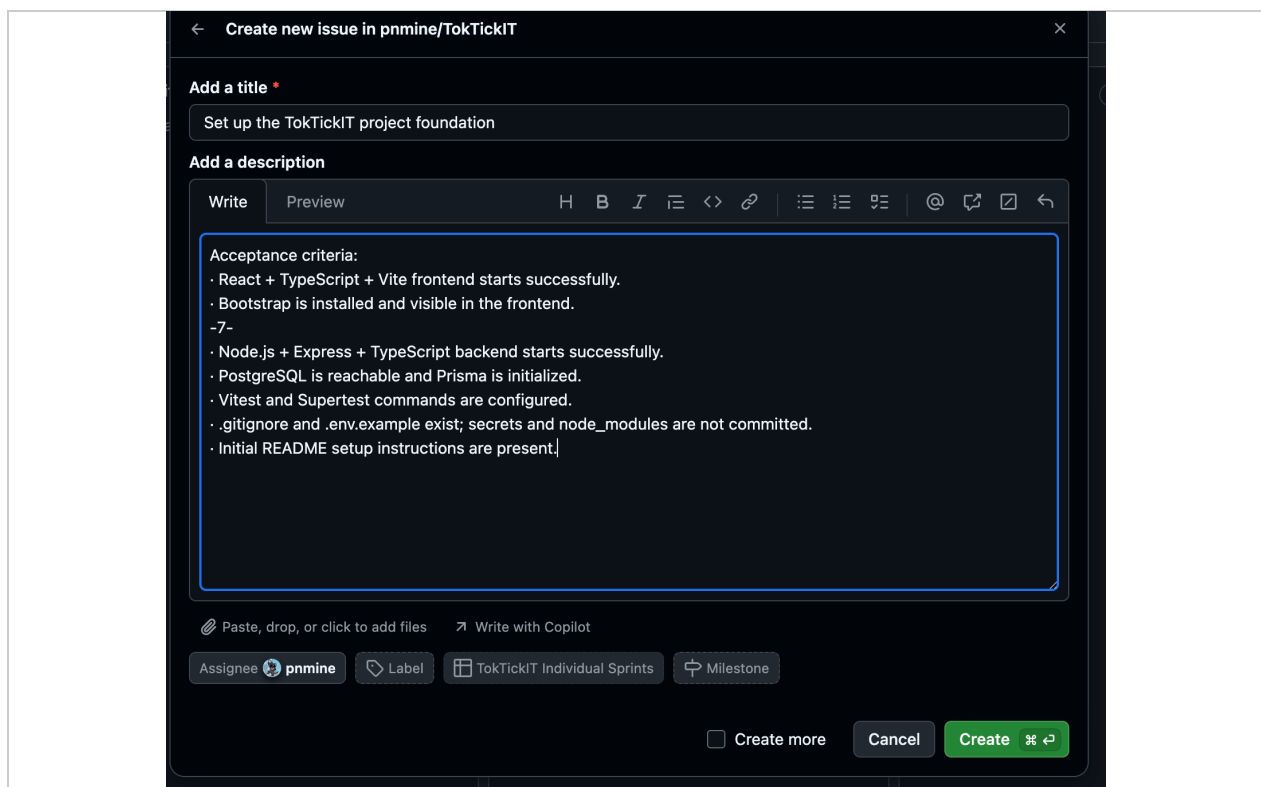
ขั้นที่ 6 เพิ่ม Issue ของคุณ ที่กั้นคอลัมน์ Backlog กด "+ Add item" แล้วเลือก "Create new issue"



Create new issue, not Create a draft. A draft is not a real Issue and cannot be linked to a PR.  
เลือก Create new issue ไม่ใช่ Create a draft เพราะ draft ไม่ใช่ Issue จริง link กับ PR ไม่ได้

Step 7. Put in the title , description and assignee, check that the project field says TokTickIT Individual Sprints, then click Create.

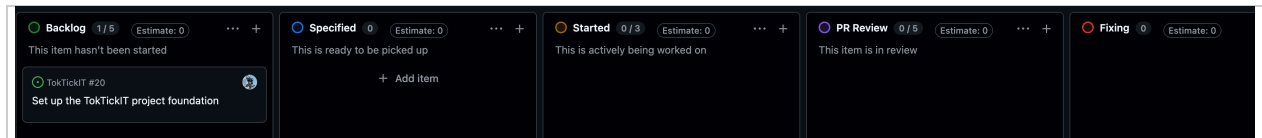
ขั้นที่ 7 ใส่หัวข้อ , คำอธิบายและผู้ที่ได้รับมอบหมาย เช็คว่าช่อง project ขึ้นว่า TokTickIT Individual Sprints แล้วกด Create



Title, description, and the project field at the bottom.  
หัวข้อ คำอธิบาย และช่อง project ที่ด้านล่าง

Step 8. Done. Every new Issue lands in Backlog, and you move it right as the work moves along.

ขั้นที่ 8 เสร็จแล้ว Issue ใหม่ทุกอันจะไปอยู่ที่ Backlog แล้วค่อยลากไปทางขวาตามความคืบหน้าของงาน



The finished board with the first Issue in Backlog.  
บอร์ดที่เสร็จแล้ว โดยมี Issue แรกอยู่ใน Backlog